CONCERT BOX OFFICE API

PREPARED FOR **Valery**

Stack —	Due —	Time —	Endpoints —
C# · ASP.NET Core · EF Core · SQLite	First class session after break	10–14 hours across the week	6 (5 CRUD + 1)

THE BRIEF

A small concert promoter books touring acts into clubs and theaters around the city and sells tickets from a laptop at the door. They want an API that lists upcoming shows, keeps two shows from being booked into the same venue on the same night, and never sells more tickets than the room holds. Every show is a **Concert**.

/01

WHAT YOU ARE BUILDING

Everything in this handout is required unless it is in the Stretch section.

Read the whole document once before writing any code.

PROJECT	`BoxOffice.Api`, an ASP.NET Core Web API using controllers
DATABASE	SQLite file `boxoffice.db` managed by EF Core migrations
TABLE	One table, `Concerts`, holding `Concert` rows. No relationships to other tables.
LAYERS	`ConcertsController` → `IConcertService` → `IConcertRepository` → `AppDbContext`
REPOSITORY	Exactly five methods: `GetAll`, `GetById`, `Create`, `Update`, `Delete`
ENDPOINTS	Six. Full CRUD on `/api/concerts` plus one domain endpoint, `PATCH /api/concerts/{id}/sell`

RULES — Four business rules enforced in the service layer, returning 400 or 409

What this practices

- Wiring the full n-tier chain for one resource and registering every piece with dependency injection.
- Keeping each layer honest: HTTP in the controller, decisions in the service, EF Core in the repository.
- Designing DTOs that differ from the entity on purpose: hidden server-set fields and one computed field.
- Choosing the right status code, especially the line between 400 and 409.
- Writing filtering and rule checks with LINQ in the service, since the repository only hands back the whole table or one row.

/02

REQUIRED ARCHITECTURE

One table, one controller, one service, one repository.

If you add a second table in the stretch goals, it gets its own copy of all four, following the same pattern.

01	→ 02	→ 03	→ 04	→ 05
Controller	**Service**	**Repository**	**DbContext**	**SQLite**
Routes, model binding, status codes. Calls the service interface. Knows nothing about EF Core.	Business rules, validation beyond attributes, mapping entity ↔ DTO, LINQ filtering. Calls the repository interface.	The only place EF Core appears. Five methods, returns entities, no DTOs, no HTTP.	`DbSet<Concert>` and the SQLite connection. Nothing else.	The `boxoffice.db` file created by migrations.

Folder layout

```
BoxOffice.Api/
├── Controllers/
│   └── ConcertsController.cs
├── Services/
│   ├── IConcertService.cs
│   └── ConcertService.cs
├── Repositories/
│   ├── IConcertRepository.cs
│   └── ConcertRepository.cs
├── Models/
│   └── Concert.cs
├── DTOs/
│   ├── CreateConcertDto.cs
│   ├── UpdateConcertDto.cs
│   └── ConcertResponseDto.cs
├── Data/
│   └── AppDbContext.cs
├── Migrations/                 generated
├── Program.cs
```

The repository contract

REPOSITORIES/ICONCERTREPOSITORY.CS

```
public interface IConcertRepository
{
    IEnumerable<Concert> GetAll();
    Concert? GetById(int id);
    Concert Create(Concert concert);
    Concert Update(Concert concert);
    bool Delete(int id);
}
```

Every repository you write this term has exactly this shape. `GetAll` is `ToList()` over the whole table. No `async` or `await` anywhere in this project; we have not covered it yet. There are no search or filter methods here on purpose: that work belongs to the service, using LINQ over what `GetAll` returns.

```
├── appsettings.json
└── README.md
```

If we put interfaces in an `Interfaces/` folder in class, do that instead. The point is that every piece exists and is named consistently.

Layer rules — REQUIRED

01	The controller depends only on `IConcertService`. It never touches the repository or `AppDbContext`.
02	The service depends only on `IConcertRepository`. It never returns an entity to the controller, only DTOs.
03	The repository is the only class that uses `AppDbContext` or any `Microsoft.EntityFrameworkCore` type.
04	All three are registered in `Program.cs` with `AddScoped`, interface to implementation.
05	`GetById` returns `null` and `Delete` returns `false` when the id does not exist. The service turns that into a 404; the repository never throws for a missing row.

/03

DATA MODEL

`string?` means nullable. Everything else is required.

Create `Models/Concert.cs` with these properties. Use the C# types shown. Simple limits can be enforced with validation attributes on the DTOs; the service handles anything that needs a comparison or a lookup.

PROPERTY	TYPE	RULES
`Id`	`int`	Primary key, generated by the database
`Artist`	`string`	Required, max 100
`OpeningAct`	`string?`	Optional, max 100
`Venue`	`string`	Required, max 100
`City`	`string`	Required, max 60
`Date`	`DateTime`	Required; must be in the future when created
`Genre`	`string`	Required, max 50
`TicketPrice`	`decimal`	Greater than 0
`Capacity`	`int`	1 through 100000
`TicketsSold`	`int`	Set to 0 by the server on create; changed only by the sixth endpoint

There are no foreign keys or navigation properties in this assignment. The table stands alone.

/04

DTOS

Validation attributes go on the DTOs, not the entity.

A failed attribute becomes an automatic 400.

Clients never see or send the `Concert` entity. Three DTO classes in `DTOs/` define the API's shape, plus one more DTO for the sixth endpoint.

`CreateConcertDto`	All fields except `Id` and `TicketsSold`. The client never sets how many tickets are sold; a new show starts at 0.
`UpdateConcertDto`	Same shape as the create DTO. `TicketsSold` changes only through the sixth endpoint, never through `PUT`.
`ConcertResponseDto`	All fields plus a computed `TicketsRemaining` (`int`) equal to `Capacity` minus `TicketsSold`.
`Sixth endpoint`	`SellTicketsDto` with a single `Quantity` integer, used as the body of the sixth endpoint.

COMPUTED `TicketsRemaining` does not exist on the entity or in the database. The service calculates it every time it maps an entity to a response DTO. That mapping should live in one private method in the service so every endpoint returns the same shape.

/05

ENDPOINTS

The sixth endpoint is the one that makes this API *this* API. Build CRUD first, then this.

Six endpoints, all on `ConcertsController` with `[ApiController]` and `[Route("api/concerts")]`. Not every endpoint can produce every error; the error column lists the ones this assignment's rules can trigger.

METHOD	ROUTE	BODY	SUCCESS	ERRORS
GET	/api/concerts	none	200 list of response DTOs	—
GET	/api/concerts/{id}	none	200 response DTO	404
POST	/api/concerts	CreateConcertDto	201 + Location header	400 · 409
PUT	/api/concerts/{id}	UpdateConcertDto	204	400 · 404 · 409
DELETE	/api/concerts/{id}	none	204	404 · 409
PATCH	/api/concerts/{id}/sell	{ "quantity": 2 }	200 with the updated concert	404 unknown id · 400 quantity under 1 · 409 not enough tickets remaining

THE SIXTH	The door sale. Add `Quantity` to `TicketsSold` and save, but only if `TicketsSold + Quantity` is still within `Capacity`. Notice the difference between 400 (asking for zero or negative tickets makes no sense) and 409 (a real request the room cannot satisfy).

Status code conventions

CODE	USE IT WHEN
`200 OK`	Successful `GET`, and the sixth endpoint when it returns the changed record
`201 Created`	Successful `POST`. Return `CreatedAtAction(nameof(GetById), new { id }, dto)` so the `Location` header is set
`204 No Content`	Successful `PUT` and `DELETE`. No body
`400 Bad Request`	Validation failed: a required field is missing, a number is out of range, a value is not one of the allowed words
`404 Not Found`	The id (or lookup value) does not exist
`409 Conflict`	The request was well-formed but a business rule says no: duplicates, a state that cannot change, a limit that is already reached

/06

BUSINESS RULES

400 the request itself is wrong.

409 the request is fine but the current state says no.

These live in `ConcertService`. The number on the right is the status code the client must receive when the rule is broken. The controller's job is only to translate the service's answer into that code.

01	On create, `Date` must be in the future. Compare against `DateTime.UtcNow` in the service.	**400**
02	One show per venue per night. Two concerts cannot share the same `Venue` (case-insensitive) on the same calendar day. Compare `Date.Date` values with `Any()` over `GetAll()`.	**409**
03	`TicketPrice` must be greater than 0 and `Capacity` must be 1 through 100000. Use validation attributes on the DTOs.	**400**
04	A sale that would push `TicketsSold` past `Capacity` is rejected. Selling exactly up to capacity is allowed.	**409**

PATTERN	Because the repository only offers `GetAll` and `GetById`, every uniqueness check, count, and filter is LINQ in the service: `var all = _repo.GetAll();` then `all.Any(...)`, `all.Count(...)`, `all.Where(...)`. The list is already in memory, so `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` works for case-insensitive comparisons.

/07

STRETCH GOALS

Stretch work only counts if the README says it exists.

Optional. Only start these once all six endpoints and all four rules work, and note in the README which ones you attempted.

Second table: Venue — STRETCH A

Add a `Venue` model (`Name` unique, `City`, `Capacity` greater than 0, `HasParking`) with its own repository, service, controller, and the same five CRUD endpoints. No relationship to `Concert`; `Venue` on a concert stays plain text. Add `GET /api/venues/search?city=`.

Upcoming — STRETCH B

`GET /api/concerts/upcoming` returns concerts with a `Date` from now forward, soonest first.

Search — STRETCH C

`GET /api/concerts/search?genre=&city=&maxPrice=` with every parameter optional and case-insensitive.